Utilización de APIs Tridimensionales
Práctica 4: Carga de mallas

En esta práctica, vamos a añadir soporte de carga de mallas en formato XML, cargando un escenario que podremos recorrer con la cámara utilizando un sistema de control típico de primera persona (movimiento con los cursores o WSAD, y rotación con el ratón).

Clase Object3D

Vamos a añadir un método a la clase para que, además de poder construir una malla proceduralmente mediante código en contructores de clases heredadas, podamos cargarla de un archivo de disco. El método será el siguiente:

```
void loadDataFromFile (std::string filename);
```

El método recibe el nombre del archivo del cual cargar la malla. Dado que la especificación de shaders en el archivo es opcional, en caso de no estar definido (puede no existir la etiqueta <shaders>), los materiales utilizarán un shader por defecto. Se puede añadir a la clase System un atributo estático de tipo “RenderProgram” que cargue al inicio los shaders por defecto “shader.vert” y “shader.frag”, y se usará en caso de no haberse especificado shaders en el archivo XML.

Para realizar la carga de la malla, consultaremos la documentación del tema para el uso de la librería **PugiXML** y la estructura utilizada en el archivo. Se debe usar la función “loadObj” vista en clase, cargando primero una clase Material con los datos del archivo “MSH”, y luego cargando los datos de mallas con loadObj. Todas las mallas que estén dentro de la misma etiqueta “buffer” compartirán el mismo Material.

Además, tenemos la función `splitString` para extraer elementos de un string (nos crea un vector del tipo especificado por el template):

```
template <typename T>
std::vector<T> splitString(const std::string& str, char delim) {
std::vector<T> elems;
std::stringstream sstream(str);
```

```
std::string item;
T tipoDato;
if (str != "") {
while (std::getline(sstream, item, delim))
{
std::istringstream str(item);
str >> tipoDato;
elems.push_back(tipoDato);
}
}
return elems;
}
```

Clase InputManager y GLFWInputManager

Definir una estructura “mouseState_t” que pueda almacenar el estado de los eventos de ratón:

```
typedef struct mouseState_t {
   double x;
   double y;
   std::map<int, bool> button;
}mouseState_t;
```

Añadir un atributo estático “mouseState” de tipo “mouseState_t” a la interfaz InputManager, y los métodos de acceso a sus atributos:

```
static bool isMousePressed(int key);
static double getMouseX();
static double getMouseY();
```

Implementar en la clase GLFWInputManager los métodos de eventos necesarios para actualizar la estructura mouseState usando GLFW.

Clase CameraFPS

En la práctica anterior se creó una cámara que permitía moverse usando las teclas de teclado. Ahora se pide implementar una cámara “libre” que imite los movimientos de un objeto en primera persona, heredera de la clase Camera:

- Teclas A,D: Movimientos de cámara en eje X positivo/negativo respectivamente
- Teclas W,S: Movimiento “hacia adelante” y “hacia atrás”. Se moverá en dirección al punto (lookAt) al que esté mirando la cámara
- Ratón: Cambiará de posición el punto al que esté mirando la cámara:

- Movimientos horizontales (ejeX) del ratón:
    - El punto de mira rotará alrededor del eje Y de la cámara
- Movimientos verticales (ejeY) del ratón
    - El punto de mira rotará alrededor del eje X de la cámara

Para ocultar el puntero y hacer más realista el efecto, usad las siguientes funciones después de haber creado la ventana de glfw:

```
glfwSetInputMode(window, GLFW_CURSOR, GLFW_CURSOR_DISABLED);
glfwSetCursorPos(window, 0.0, 0.0);
```

Programa principal

Realizaremos un nuevo programa en el que crearemos un Objeto3D cuyo constructor cargará la malla **“data/asian_town.msh”**. Crearemos una cámara de tipo CameraFPS y un modelo con la malla cargada, y añadiremos ambos al mundo. Al pulsar las teclas de cursor arriba y cursor abajo, la cámara se moverá hacia delante y hacia atrás en la dirección en la que mira. Al pulsar los cursores izquierda y derecha, se moverá en la dirección correspondiente. Al mover el ratón en horizontal, la cámara mirará a izquierda y derecha, y al moverlo en vertical, mirará arriba y abajo. Consultar la documentación del tema para resolver esta parte.